BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 2

Roll no: 56 PRN No: 12411305 Name of Student: Anuja Dhiraj Kumbhar

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix

Algorithm :

1. Import the required libraries.
2. Load the Wine (or Iris) dataset.
3. Split the data into training and testing sets.
4. Scale the input features.
5. Create the MLP classifier.
6. Train the model using the training data.
7. Predict the test data.
8. Find the accuracy and confusion matrix.
9. Display the results and evaluate the model.

Code :

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
wine=load_wine()
X,y=wine.data,wine.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_st
ate=42)
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
mlp=MLPClassifier(hidden_layer_sizes=(10,10),activation='relu',solver='ada
m',max_iter=1000,random_state=42)
mlp.fit(X_train,y_train)
y_pred=mlp.predict(X_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
print("\nClassification Report:\n",classification_report(y_test,y_pred))
import matplotlib.pyplot as plt
import seaborn as sns
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=wine.target_names,
    yticklabels=wine.target_names
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix Heatmap")

plt.show()
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(mlp.loss_curve_, linewidth=2)
```

```
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)

plt.show()
import pandas as pd
comparison = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred
})

print(comparison.head(15))

comparison.head(15).plot(
    kind='bar',
    figsize=(10,5)
)

plt.title("Actual vs Predicted")
plt.ylabel("Wine Class")
plt.show()
from sklearn.metrics import classification_report
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

report = classification_report(y_test, y_pred, output_dict=True)

report_df = pd.DataFrame(report).transpose()
report_df = report_df.iloc[:3, :3]

plt.figure(figsize=(6,4))

sns.heatmap(
    report_df,
    annot=True,
    cmap="YlGnBu",
    fmt=".2f",
    linewidths=0.5
)
plt.title("Classification Report Heatmap")
plt.xlabel("Metrics")
plt.ylabel("Wine Classes")
plt.show()
```

Output :

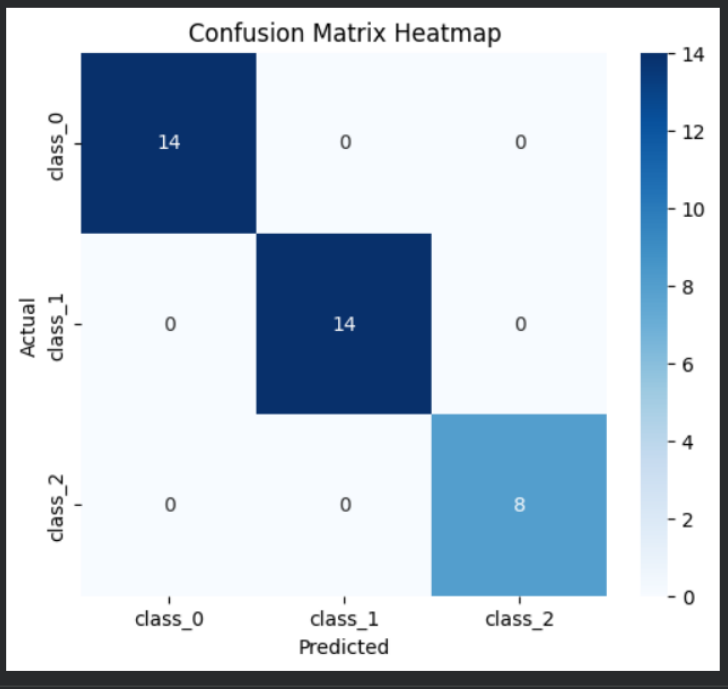
- **Confusion matrix**

```
[6]
✓ 0s
mlp=MLPClassifier(hidden_layer_sizes=(10,10),activation='relu',solver='ad
mlp.fit(X_train,y_train)
y_pred=mlp.predict(X_test)
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
print("\nClassification Report:\n",classification_report(y_test,y_pred))
```

```
Accuracy: 1.0
Confusion Matrix:
 [[14  0  0]
 [ 0 14  0]
 [ 0  0  8]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        14
           1       1.00      1.00      1.00        14
           2       1.00      1.00      1.00         8

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36
```

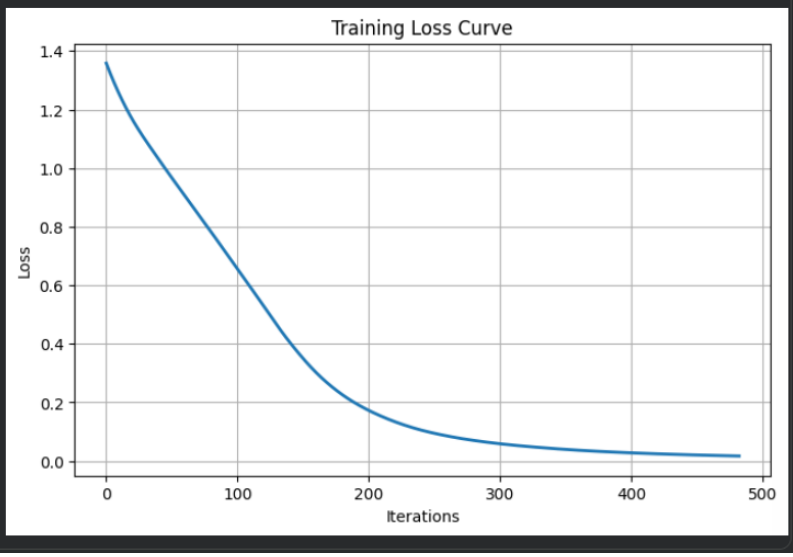
- **Confusion matrix heatmap**

```
[9]
✓ 0s
plt.title("Confusion Matrix Heatmap")

plt.show()
```



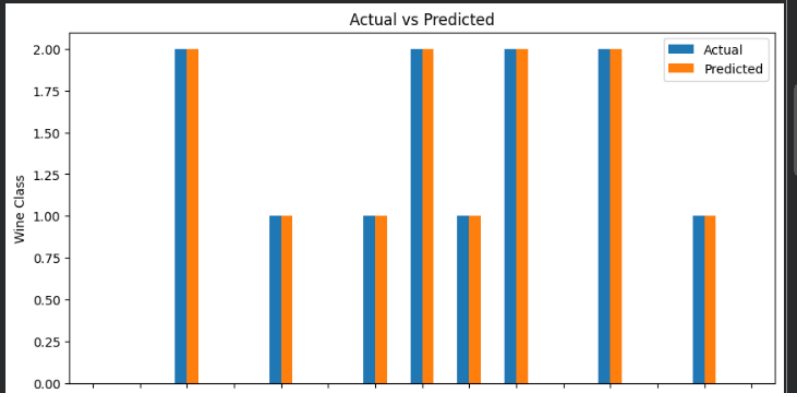
- **Training loss curve**

```
plt.grid(True)

plt.show()
```



- **Actual vs predicted comparison**

```
    Actual  Predicted
0        0          0
1        0          0
2        2          2
3        0          0
4        1          1
5        0          0
6        1          1
7        2          2
8        1          1
9        2          2
10       0          0
11       2          2
12       0          0
13       1          1
14       0          0
```



- **Classification report**

```
)

plt.title("Classification Report Heatmap")
plt.xlabel("Metrics")
plt.ylabel("Wine Classes")

plt.show()
```